



TWITCH ANALYTICS

Las Grecas

ÍNDICE

1. Introducción.....	3
2. Objetivos.....	3
3. Consideraciones (probar implementación y dónde se encuentra alojada).....	4
4. Detalles sobre implementación.....	5
5. Dificultades encontradas (problemas y decisiones tomadas para solucionarlo).....	8

1. Introducción

En la actualidad, Twitch se ha consolidado como la plataforma de streaming en vivo más relevante en el ámbito de los videojuegos y los eSports. Desde su lanzamiento en 2011, su crecimiento ha sido exponencial, alcanzando audiencias globales y generando ingresos millonarios. La plataforma ofrece a los creadores de contenido la oportunidad de interactuar en tiempo real con sus comunidades, mientras que los espectadores pueden consumir contenido en vivo de sus streamers favoritos.

Con la creciente popularidad de Twitch, la cantidad de datos generados en la plataforma es masiva. Sin embargo, muchas marcas, creadores de contenido y agencias tienen dificultades para interpretar y aprovechar estos datos de manera efectiva. En este contexto nace el proyecto **Twitch Analytics**, cuyo objetivo es procesar y transformar estos datos en información útil.

2. Objetivos

El objetivo principal del proyecto Twitch Analytics es desarrollar un servicio basado en la API de Twitch que permita acceder, procesar y presentar información relevante sobre los streamers y los streams en vivo. A través de la implementación de endpoints específicos, el sistema proporcionará métricas clave que pueden ser utilizadas para:

1. Obtener información detallada de un streamer de Twitch: Permitir a los usuarios consultar datos sobre un streamer específico a través de su ID, obteniendo detalles como su nombre, imagen de perfil, tipo de cuenta y fecha de creación.
2. Consultar streams en vivo: Proporcionar una lista actualizada de los streams que están activos en la plataforma, facilitando la visualización de contenido en tiempo real.
3. Consultar los streams más populares con datos enriquecidos: Ordenar y mostrar los streams con mayor audiencia, incluyendo información adicional sobre cada streamer, como su nombre de usuario y su imagen de perfil.
4. Garantizar accesibilidad y facilidad de uso: La API desarrollada debe ser pública y accesible desde cualquier sitio, cumpliendo con el requisito de no depender de servidores internos de la universidad.

Este proyecto ha sido desarrollado en PHP y se ha documentado detalladamente en el presente informe, abordando las decisiones tomadas, los desafíos encontrados y las instrucciones para su implementación y prueba.

3. Consideraciones (probar implementación y dónde se encuentra alojada)

Para alojar nuestra aplicación, comenzamos investigando cómo configurar un servidor y seguimos una serie de pasos:

En primer lugar, creamos una instancia en Google Cloud con Ubuntu, PHP y MySQL, lo que nos permitió gestionar tanto la base de datos como el entorno de desarrollo.

Posteriormente, configuramos el subdominio **twitchanalytics.42web.io** con **Infinity Free**. Sin embargo, este servicio no permitía modificar el registro a tipo A para asociarlo a una IP específica. Para resolver esta limitación, recurrimos a un servicio de DNS gratuito a través de **freeDNS**.

Gracias a esta configuración, logramos que nuestra página web fuera accesible en la siguiente dirección: **twitchanalytics.42web.io.crabdance.com**.

En dicho archivo, se ha agregado una regla de reescritura para gestionar todas las solicitudes dirigidas a <http://34.56.178.39>.

En lugar de utilizar .htaccess, se decidió usar un archivo "router.php" que actuará como intermediario, recibiendo las solicitudes por parte del cliente y redirigiéndolas a los archivos con extensión .php. Esto para evitar mostrar la extensión .php en el navegador.

Para ello se ha añadido:

```
<VirtualHost *:80>
  ServerName tu-dominio.com
  DocumentRoot /var/www/html

  <Directory "/var/www/html">
    AllowOverride All
    Require all granted
  </Directory>

  RewriteEngine On
  RewriteCond %{REQUEST_FILENAME} !-f
  RewriteRule ^(.*)$ router.php [QSA,L]
</VirtualHost>
```

4. Detalles sobre implementación

config.php

El archivo config.php tiene un propósito clave dentro del proyecto: gestionar de manera segura las credenciales de acceso a la API de Twitch, evitando que información sensible como el CLIENTE_ID y el CLIENTE_SECRET estén directamente expuestas en el código fuente.

1. Uso de Variables de Entorno:

El código implementa dos estrategias para obtener las credenciales:

- Desde un archivo .env, si existe en el directorio del proyecto.
- Desde las variables de entorno del sistema, si no se encuentra el archivo .env.

2. Carga de Variables desde .env

```
if (file_exists(__DIR__ . '/.env')) {  
    $env = parse_ini_file(__DIR__ . '/.env');  
    define('CLIENT_ID', $env['CLIENT_ID'] ?? '');  
    define('CLIENT_SECRET', $env['CLIENT_SECRET'] ?? '');  
}
```

- ☐ file_exists(__DIR__ . '/.env'): Verifica si el archivo .env está presente en el mismo directorio.
- ☐ parse_ini_file(__DIR__ . '/.env'): Carga las variables definidas en el archivo .env.
- ☐ define('CLIENT_ID', \$env['CLIENT_ID'] ?? ''): Si la variable existe en .env, la asigna; de lo contrario, usa una cadena vacía. Mismo procedimiento para CLIENT_SECRET.

3. Carga de Variables desde el Entorno del Sistema

Si no existe un archivo .env, se intentan cargar las credenciales desde el sistema:

```
} else {  
    define('CLIENT_ID', getenv('CLIENT_ID'));  
    define('CLIENT_SECRET', getenv('CLIENT_SECRET'));  
}
```

- ☐ getenv('CLIENT_ID') y getenv('CLIENT_SECRET') buscan las variables en el entorno del sistema.

4. Validación de Credenciales

```
if (!CLIENT_ID || !CLIENT_SECRET) {  
    die("Error: CLIENT_ID o CLIENT_SECRET no están configurados. Asegúrate de definirlos en .env o variables de entorno.");  
}
```

- ☐ Si alguna de las variables no está configurada correctamente, se detiene la ejecución con un mensaje de error.

Este enfoque mitiga vulnerabilidades como CWE-798 (uso de credenciales codificadas en el código fuente), al almacenar las claves en un archivo externo en lugar de dentro del código.

get_token_test.php

El archivo `get_token_test.php` tiene como propósito obtener un token de acceso de la API de Twitch mediante el protocolo OAuth 2.0, utilizando las credenciales de la aplicación (Client ID y Client Secret). Este token es necesario para realizar futuras solicitudes a la API de Twitch.

El script gestiona de manera eficiente el almacenamiento y reutilización del token, evitando solicitudes innecesarias hasta que expire.

1. Verifica si ya existe un token válido en `token.json`.
2. Si el token es válido, lo reutiliza.
3. Si no hay token o ha expirado, solicita uno nuevo a la API de Twitch.
4. Guarda el nuevo token en `token.json` con su fecha de expiración.
5. Maneja errores en casos de fallos en la solicitud o credenciales incorrectas.

get_top_streams.php

El archivo `get_top_streams.php` implementa un endpoint en PHP que interactúa con la API de Twitch para proporcionar un listado JSON con los datos de los primeros N (limit) streams en vivo más populares en Twitch, ordenados por número de espectadores (`viewer_count`) y enriquecidos con información adicional sobre los streams.

info_live_streams.php

El archivo `info_live_streams.php` es un script en PHP diseñado para consultar y obtener información en tiempo real sobre transmisiones en vivo en la plataforma de Twitch, utilizando la API oficial de Twitch. Su implementación sigue buenas prácticas como la reutilización de un token de autenticación, el manejo de errores y la respuesta en formato JSON estructurado.

info_streamer.php

El archivo `info_streamer.php` tiene como objetivo consultar información de un streamer en Twitch mediante la API oficial de Twitch. Para ello, requiere un identificador único (id) del usuario, el cual se proporciona como parámetro en la URL o en la línea de comandos.

prueba.php

No es imprescindible para el proyecto, pero se usó para comprobar los permisos de lectura/escritura sobre el archivo `token.json`

router.php

Este programa actúa como un **enrutador personalizado**, que intercepta las solicitudes de URL y las dirige a los archivos PHP correspondientes según la ruta solicitada. Si la ruta solicitada corresponde a un archivo físico, se devuelve directamente; de lo contrario, se ejecuta el código PHP apropiado o se muestra un error 404.

5. Dificultades encontradas (problemas y decisiones tomadas para solucionarlo)

Problema 1: Modificación del archivo php.ini

Para realizar solicitudes seguras a través de HTTPS hay que habilitar las extensiones curl y openssl en php, a través del archivo de configuración php.ini (puede aparecer como php.ini-development, en este caso se hace una copia pero sin -development)

```
extension=curl;  
extension=openssl;
```

Problema 2: Problemas en la función: `curl_setopt($ch, CURLOPT_SSL_VERIFYPEER, true);`

Al realizar solicitudes cURL en PHP, nos enfrentamos con un problema relacionado a verificación SSL, que estaba fallando. Para solucionar el problema inicialmente se estableció el segundo parámetro con valor false, pero debido a que eso es inseguro, se descargó el archivo cacert.pem (contiene certificados de autoridades de certificación (CA) de confianza, se ha extraído de Mozilla) y se modificó el archivo de configuración de php (php.ini) indicando la ruta de ese archivo en: `curl.cainfo cacert.pem`.

Problema 3: Firewall

Al intentar acceder a Google u otros servicios web, la conexión no era posible. Esto se debe a que el firewall bloqueaba el tráfico entrante en el puerto 80, el cual es utilizado por el protocolo HTTP para permitir solicitudes web sin cifrado.

El firewall es un sistema de seguridad de red que controla y restringe el tráfico según reglas predefinidas. En este caso, una de esas reglas faltaba y estaba impidiendo la comunicación entre su sistema y los servidores de Google.

Problema 4: Naming:

Al finalizar el proyecto, nos dimos cuenta de que las consultas (queries) no coincidían con las especificadas en los requisitos. Por ello, tuvimos que modificar la estructura del proyecto para ajustarla correctamente.

Problema 5: Servidor:

Hemos tenido que reemplazar el servidor, ya que otorgar permisos en exceso provocó su fallo. Para solucionar el problema, hemos reiniciado el proceso utilizando una nueva IP.

Problema 6: Código con comentarios que sobran:

No hemos logrado refactorizar el código de manera eficiente, y actualmente contiene numerosos comentarios redundantes. Hemos intentado considerar los nombres de los archivos y asegurarnos de que las respuestas a las solicitudes GET cumplan con las especificaciones del documento.